EXPERIMENT 3 – AGILE PROJECT MANAGEMENT WITH GIT

Detailed practical steps | Both ways of completing the experiment

Aim: Create a GitHub Projects board to manage tasks generated by scheduled jobs, map cron/systemd tasks to user stories or issues, track task status, and link commits to Agile tasks.

Important: There are two ways to do the practical. Option A works independently and does not require script files. Option B reuses scripts created in Experiment 1/2. You do not need to do both options; choose the one that matches your lab requirement.

PART 1 – Common setup: Create the GitHub Project board

Step 1. Open GitHub
1. Sign in at github.com.
2. Open your repository, for example **AutomationProject**.
3. Select the **Projects** tab. If you do not see it, open the repository menu or your profile's Projects area.

Step 2. Create a project
1. Click **New project**.
2. Choose the **Board** layout.
3. Enter the name **Automation Task Tracker**.
4. Create the project.

Step 3. Set up the status columns
Use the default Status field if available. Create or rename the options so the board has these four statuses:

```
Backlog → To Do → In Progress → Done
```

If the board initially shows different columns, use the Status field menu to edit its options. Keep the four statuses above.

Step 4. Add three task cards
In the Backlog column, click **+ Add item** and create these cards:

```
Backup automation using cron
Automate Git commit task
System report generation
```

A project card can be a draft item or a linked GitHub issue. For this experiment, either is enough to show the tasks on the board. If you want to link commits to a task using an issue number, create/link the task as a GitHub Issue.

PART 2 – OPTION A: Complete Experiment 3 without creating scripts

Use this option if you are doing Experiment 3 independently. The experiment asks you to map scheduled jobs to Agile tasks; it does not require you to create or execute the scripts again.

Step A1. Open the first task card
Click **Backup automation using cron**. Add this information in its description or notes:

```
Mapped job: Automated backup
Script name: backup.sh
Schedule: Cron job
Purpose: Back up selected files automatically
```

Step A2. Add details to the second card
Open **Automate Git commit task** and add:

```
Mapped job: Automatic Git commit and push
Script name: auto_commit.sh
Schedule: Scheduled job (cron)
Purpose: Add, commit and push changed files to GitHub
```

Step A3. Add details to the third card
Open **System report generation** and add:

```
Mapped job: System information report
Script name: system_report.sh
Schedule: Cron job
Purpose: Generate disk, memory and CPU usage report
```

Step A4. Save each card
Press Enter or click outside the description field if GitHub autosaves the draft item. For an Issue, click **Save** or **Submit new issue** as shown.

What you have done: You have documented which scheduled job/script corresponds to each Agile task. The script names here are labels for the mapping; this option does not create the files.

PART 3 – OPTION B: Complete Experiment 3 by reusing scripts from Experiments 1 and 2

Use this option if you already completed the earlier experiments and the script files still exist. Do not recreate them unless they are missing.

Step B1. Check which scripts are available
Open the terminal and run:

```
ls ~/scripts
ls ~/AutomationProject
```

Look for the scripts you previously created. Typical locations from the earlier exercises are:

```
~/scripts/backup.sh
~/AutomationProject/auto_commit.sh
~/scripts/system_report.sh
```

Your actual paths may differ. If a file is not listed, do not assume it exists; use Option A for that task or create the missing script only if your instructor requires execution.

Step B2. Map the existing files to the task cards
Open each card on GitHub Projects and record the actual script path:

```
Backup automation using cron
Script: ~/scripts/backup.sh

Automate Git commit task
Script: ~/AutomationProject/auto_commit.sh

System report generation
Script: ~/scripts/system_report.sh
```

If your script is stored in a different folder, enter the path shown by your terminal instead.

Step B3. (Optional) Verify a script exists and is executable
Run the relevant commands only for scripts that exist:

```
ls -l ~/scripts/backup.sh
ls -l ~/AutomationProject/auto_commit.sh
ls -l ~/scripts/system_report.sh
```

If a script exists but does not have execute permission, use:

```
chmod +x /actual/path/script_name.sh
```

Step B4. (Optional) Check the cron schedule
Display scheduled jobs:

```
crontab -l
```

If the matching cron entry exists, mention its schedule in the task description. Do not add another cron entry just for this mapping step.

PART 4 – Move tasks through Agile statuses

For either option, use the board to show task progress. Drag each card from one column to the next, or change its Status field:

```
Backlog → To Do → In Progress → Done
```

Example: keep a task in Backlog before work begins; move it to To Do when ready; move it to In Progress while setting up/checking the job; move it to Done after successful completion. For a task that fails, leave it In Progress or move it back to To Do until fixed.

PART 5 – Link Git commits to Agile tasks (when using GitHub Issues)

If your task is a GitHub Issue, note its issue number (for example, #3). In the terminal, from inside the Git repository, commit with the issue number in the message:

```
cd ~/AutomationProject
git add .
git commit -m "Complete automation task #3"
git push
```

Replace #3 with the actual issue number. The issue reference in the commit message helps connect the code change to the task. If you created only draft project cards and not Issues, you can still track tasks on the board, but there is no issue number to reference.

PART 6 – Final verification

```
git status
git log --oneline
crontab -l
```

Then return to GitHub Projects and verify that all three task cards are present and are in the correct status columns. If you used Issues, open the issue and check its linked project/commit activity where available.

What to write as the result

A GitHub Projects board was created to manage scheduled-job tasks. The backup, automatic Git commit, and system report tasks were mapped to their corresponding scripts/jobs. Task progress was tracked using Backlog, To Do, In Progress, and Done statuses. Git commits were linked to issues where applicable to maintain traceability.

Quick exam reminder

Option A: Board + task cards + descriptions. No script files required.
Option B: Reuse the existing scripts from Experiments 1/2, record their actual paths, and verify the cron schedule if required.
Both options: Create the board, add tasks, map jobs, update statuses, and show commit-to-task traceability if using Issues.